

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחיינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב - מואץ
67130 – מבוא למדעי המחשב - רגיל
67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 15.3.2013
משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' – תשע"ג – 2012-2013
פרופ' דני לישצ'ינסקי, מר ינאי גונצ'רובסקי

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (34 נק'), אך הציון המקסימלי הוא 100.
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחיינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית), אין צורך לכתוב javadoc.
- פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
- בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

בהצלחה!

שאלה 1 (34 נקודות)

א. (8 נקודות) כתבו את השיטה:

```
public static int extractDigit(int num, int pos)
```

המקבלת מספר שלם אי-שלילי `num` ומספר שלם אי-שלילי `pos` ומחזירה את הספרה ה-`pos` שלו בייצוג עשרוני (עבור `pos=0` תוחזר ספרת האחדות, עבור `pos=1` תוחזר ספרת העשרות וכו'). לדוגמא:

```
extractDigit(876, 0) => 6
extractDigit(876, 1) => 7
extractDigit(876, 2) => 8
extractDigit(876, 3) => 0
```

ב. (14 נקודות) השתמשו בשיטה `extractDigit` על מנת לכתוב את השיטה:

```
public static void sortByDigit(int[] arr, int pos)
```

המקבלת מערך של מספרים שלמים אי-שליליים וממיינת אותו בסדר עולה לפי הספרה ה-`pos` (בייצוג עשרוני). כלומר, משנה את סדר המספרים בו כך שראשית יופיעו כל המספרים שהספרה ה-`pos` שלהם היא 0 (אם יש כאלה), לאחר מכן יופיעו כל המספרים שהספרה ה-`pos` שלהם היא 1 (אם יש כאלה), וכו' (אחרונים יופיעו כל המספרים שהספרה ה-`pos` שלהם היא 9, אם יש כאלה).

הנחיות והבהרות:

- על השיטה להיות יציבה, כלומר מספרים אשר להם אותה הספרה ה-`pos` יופיעו במערך לאחר הפעלת השיטה באותו הסדר (ביניהם) בו הם הופיעו לפני הפעלת השיטה. לדוגמא, לאחר הרצת הקוד הבא:

```
int[] arr = {543, 246, 368, 753, 412, 847, 246, 12, 5};
sortByDigit(arr, 1);
```

המערך `arr` יכיל את המספרים הנ"ל ממויינים לפי ספרת העשרות, משמאל לימין:
5, 412, 12, 543, 246, 847, 246, 753, 368
- שימו לב כי בשל תכונת היציבות, המספרים 246, 847 ו-246, אשר להם אותה ספרת עשרות (4), מופיעים לאחר הפעלת השיטה באותו הסדר בו הם הופיעו לפני הפעלת השיטה (ראשית 543, לאחר מכן 246, לאחר מכן 847, ולאחר מכן 246 נוסף).
- על השיטה לרוץ בסיבוכיות $O(N)$ והיא רשאית להשתמש ב- $O(N)$ זכרון, כאשר N הוא מס' האיברים ב-`arr`.
- השתמשו במחלקה `Queue` (הניחו שהיא כתובה כבר), המממשת תור של מספרים, לפי ה-API המפורט להלן. הניחו שכל אחת מן השיטות של `Queue` רצה בסיבוכיות $O(1)$. הנכם רשאים להקצות מספר קבוע של תורים, ולהחזיק בכל אחד מהם לכל היותר N מספרים, כאשר N הוא מספר האיברים ב-`arr`.

```
public class Queue {
    public Queue(); // בונה תור ריק
    public void enqueue(int num); // מוסיפה את הערך הנתון לאחורי התור
    public int dequeue(); // מסירה את האיבר הראשון בתור ומחזירה אותו
    public boolean isEmpty(); // האם יש איברים בתור?
}
```

ג. (3 נקודות) נמקו מדוע השיטה שכתבתם בסעיף ב' אכן רצה בסיבוכיות $O(N)$, כאשר N הוא מס' האיברים ב-`arr`.

ד. (6 נקודות) השתמשו בשיטה `sortByDigit` על מנת לכתוב את השיטה
`public static void digitSort(int arr[], int maxDigits)`

המקבלת מערך של מספרים שלמים אי-שליליים, אשר לכל אחד מהם לכל היותר `maxDigits` (נתון) ספרות בייצוג עשרוני, וממיינת את המערך בסדר עולה.

ה. (3 נקודות) מה סיבוכיות השיטה שכתבתם בסעיף ד', כתלות באורך המערך וב-`maxDigits`? נמקו!

שאלה 2 (34 נקודות)

תזכורת:

"עץ חיפוש בינארי" הינו עץ בינארי שבו לכל קדקוד, כל ערכי הקדקודים בתת-העץ השמאלי שלו קטנים או שווים לערך שלו, וכל ערכי הקדקודים בתת-העץ הימני שלו גדולים ממנו.

"רשימה מקושרת דו-כיוונית מעגלית" הינה רשימה מקושרת שבה כל קדקוד מצביע על הקדקוד שלפניו ועל הקדקוד שאחריו. בנוסף הקדקוד הראשון מצביע על הקדקוד האחרון כעל הקדקוד שלפניו, והקדקוד האחרון מצביע על הקדקוד הראשון כעל הקדקוד שאחריו.

בשאלה זו תממשו כמה שיטות מן המחלקה **TreeList** המסוגלת לייצג הן עץ חיפוש בינארי והן רשימה מקושרת דו-כיוונית מעגלית. למחלקה זו ישנן, בין השאר, שיטות שיוודעות להמיר עץ חיפוש בינארי לרשימה דו-כיוונית מעגלית (ממוינת), ולהיפך: להמיר רשימה דו-כיוונית מעגלית (לאו דוקא ממוינת) לעץ חיפוש בינארי. המחלקה עושה שימוש בעובדה שניתן לממש הן קדקוד בעץ בינארי והן קדקוד ברשימה דו-כיוונית באמצעות אותם שלושה שדות: שדה **data**, ושני מצביעים לקדקודים אחרים במבנה הנתונים (שני הבנים במקרה של עץ והקדקודים שלפני ושאחרי במקרה של רשימה). מימוש חלקי של המחלקה נתון להלן:

```
public class TreeList {

    private final int data;
    private TreeList left;
    private TreeList right;

    // בנאי המקבל ערך והאם הקדקוד הוא קדקוד של רשימה או של עץ
    public TreeList(int data, boolean isList) {
        this.data = data;
        this.left = this.right = (isList ? this : null);
    }

    // שיטה המקבלת מצביע לעץ חיפוש בינארי וכן קדקוד חדש, ומוסיפה את הקדקוד כעלה בעץ
    // במיקום היחיד שיגרום לעץ להישאר עץ חיפוש חוקי (מבלי לשנות שאר העץ)
    // שימו לב כי שיטה זו כבר ממומשת עבורכם ואין צורך לממשה
    public static void insertTree(TreeList tree, TreeList newNode) { ... }

    // שיטות נוספות שעליכם לממש. ראו את המשך השאלה
}
```

עליכם לממש מספר שיטות סטטיות נוספות כחלק מן המחלקה הנ"ל, כמפורט בסעיפים ב'-ד'. שימו לב שבכל הסעיפים בשאלה זו עץ ריק (או רשימה ריקה) מיוצגים פשוט על ידי מצביע מטיפוס **TreeList** שערכו הוא **null**, ויש לטפל גם במקרה הזה.

א. (6 נקודות) ציירו את העץ המתקבל לאחר שימוש בשיטה **insertTree** לאחר רצף הקריאות הבא:

```
TreeList root = new TreeList(4, false);
insertTree(root, new TreeList(2, false));
insertTree(root, new TreeList(3, false));
insertTree(root, new TreeList(1, false));
insertTree(root, new TreeList(7, false));
insertTree(root, new TreeList(5, false));
insertTree(root, new TreeList(8, false));
```

מה היא הסיבוכיות הכוללת לאחר N קריאות (כאשר מתחילים מעץ ריק) לשיטה `insertTree` עם ערכים שונים? הבחינו בין המקרה הטוב ביותר לרע ביותר.

ב. (6 נקודות) ממשו שיטה המקבלת מצביע לקדקוד ברשימה דו-כיוונית מעגלית (לאו דווקא ממוינת) וממירה את הרשימה לעץ חיפוש בינארי מבלי ליצור קדקודים חדשים. השיטה מחזירה את שורש העץ המתקבל:
`public static TreeList listToTree(TreeList list)`

ג. (6 נקודות) ממשו שיטה המקבלת מצביעים לראשן של שתי רשימות שונות ומצרפת אותן לרשימה דו-כיוונית מעגלית אחת, כך שאיברי הרשימה השנייה יופיעו אחרי איברי הראשונה (אין ליצור קדקודים חדשים). השיטה מחזירה מצביע לראש הרשימה המאוחדת:
`public static TreeList joinLists(TreeList list1, TreeList list2)`

ד. (12 נקודות) ממשו שיטה המקבלת מצביע לעץ חיפוש בינארי וממירה את העץ לרשימה דו-כיוונית מעגלית ממוינת מהקטן לגדול (למעט ההצבעות בין האיברים הראשון והאחרון), מבלי ליצור קדקודים חדשים. השיטה מחזירה מצביע לראש הרשימה:
`public static TreeList treeToList(TreeList tree)`

במימוש שיטה זו מומלץ להשתמש בשיטה `joinLists` מהסעיף הקודם. מומלץ להשתמש ברקורסיה.

ה. (4 נקודות) מהי הסיבוכיות של כל אחת מן השיטות הנ"ל כפונקציה של מספר הקדקודים במבנה הנתונים בעת הפעלת השיטה? הבחינו בין המקרה הגרוע ביותר לבין המקרה הטוב ביותר.
מה יקרה אם נריץ את השיטה `treeToList` ומיד לאחר מכן את השיטה `listToTree`?
איך תושפע הסיבוכיות של שאר השיטות שיקראו לאחר מכן?

שאלה 3 (34 נקודות)

נתונה המחלקה הבאה (אין צורך לממש אותה):

```
public class SortedList {
    public SortedList(); // creates an empty list
    public void add(int num); // adds num to the list,
                               while maintaining the sorted order
    public int find(int num); // returns the index of num in the list,
                               or -1 if not found
    public int get(int i); // returns the value at the specified
                           index in the list, in O(1)
    public int size(); // returns the size of the list, in O(1)
}
```

והממשק הבא:

```
public interface ListIterator {
    /** Returns true if the iterator has more elements. */
    public boolean hasNext();
    /** Returns the next element and advances the iterator. */
    public int next();
    /** Returns the next element without advancing the iterator. */
    public int peek();
}
```

א. (6 נקודות) ממשו שיטה סטטית המקבלת אובייקט מטיפוס `ListIterator` ומדפיסה את כל הערכים שהוא מחזיר, לפי סדר החזרתם (בכל פורמט שתמצאו):

```
public static void print(ListIterator iter)
```

ב. (8 נקודות) כתבו את המחלקה `SortedListIterator` המממשת את הממשק הנ"ל על `SortedList`, כלומר מגדירה איטרטור העובר על איברי רשימה ממוינת (מיוצגת על ידי אובייקט מטיפוס `SortedList` שניתן כארגומנט לבנאי של האיטרטור). ניתן להניח כי הרשימה לא תשתנה בזמן שהאיטרטור עובר על איבריה.

ג. (8 נקודות) ממשו מחלקה בשם `MergeIterator` המממשת את הממשק `ListIterator`. הבנאי שלה מקבל שני אובייקטים מטיפוס `SortedListIterator` ומייצגת איטרטור על המיזוג (הממויין) של הרשימות המתאימות. המימוש של הבנאי צריך לעבוד בזמן $O(1)$, ושאר המימוש צריך להיות פשוט ויעיל ככל האפשר. ציינו את הסיבוכיות של כל שיטה.

ד. (8 נקודות) ממשו את המחלקה `SortedSet` המייצגת קבוצת מספרים ממוינת, באמצעות ירושה מ-`SortedList`. קבוצה ממוינת (`SortedSet`) היא סוג של רשימה ממוינת (`SortedList`) שבה כל ערך מופיע לכל היותר פעם אחת. לפיכך קריאה ל-`add` לא עושה דבר אם המספר שרוצים להוסיף כבר נמצא בקבוצה. על המימוש לכלול את שורת ההגדרה של המחלקה, בנאי המקבל מערך מספרים מטיפוס `int`, וכל שיטה נוספת של `SortedList` אשר יש צורך בהגדרתה מחדש (`overriding`). בנוסף יש להוסיף את השיטה הבאה:

```
public boolean contains(int element)
```

המחזירה אמת אם ורק אם `element` נמצא בתוך קבוצת המספרים.

ה. (4 נקודות) נניח שיש לנו שתי קבוצות ממוינות `SortedSet`: אחת `a` המכילה את המספרים {1,2,3} ואילו השנייה `b` המכילה {2,4,6}. מה ידפיס הקוד הבא (מדובר באותה שיטת `print` מסעיף א):

```
print(new MergeIterator(new SortedListIterator(a),
                        new SortedListIterator(b)));
```

שאלה 4 (34 נקודות)

א. (7 נקודות) הסבר מה הקוד הבא עושה עבור קלט $a < 0$:

```
public static String mystery(int a){
    return mysteryHelper(a, "");
}

public static String mysteryHelper(int a,String b){
    if (a == 0){
        return b;
    }
    b = (a%2)+b;
    a = a/2;
    return mysteryHelper(a,b);
}
```

ב. (3 נקודות) כתבו את הפלט (בלבד) של הקריאה `mystery(13)`.

ג. (8 נקודות) כתבו את השיטה הרקורסיבית:

```
public static void printReverse()
```

הקוראת מספרים חיוביים מן הקלט. כל עוד המספרים בסדר עולה (כלומר כל מספר גדול או שווה לקודמו) הפונקציה ממשיכה לקרוא מספרים. כאשר הסדר מופר (התקבל מספר קטן ממש מהמספר הקודם) הפונקציה מדפיסה את כל המספרים בסדר הפוך.

שימו לב - בסעיף זה אסור להשתמש בלולאות או במערכים.

יש להשתמש בשיטות הבאות (אין צורך לממש):

- שיטה הקוראת ומחזירה את המספר הבא מן הקלט:

```
public static int read()
```

- שיטה המדפיסה את המספר הנתון:

```
public static void print(int n)
```

ד. (2 נקודות) הסבר מה היית משנה בשיטה שכתבת בסעיף ג' על מנת להדפיס את המספרים על פי סדר הקבלה ולא בסדר הפוך? הסבר באורך לכל היותר 20 מילים, גם בסעיף זה אין להשתמש בלולאות, מערכים או כל מבני נתונים אחר.

ה. (14 נקודות) כתבו שיטה רקורסיבית

```
public static void minimalDiff(int[] arr, boolean[] fairDivision)
```

המקבלת מערך של מספרים שלמים ומערך של בוליאנים ומחלקת את המערך של המספרים לשתי קבוצות זרות של מספרים אשר איחודן הוא המערך המקורי, וההפרש (בערך מוחלט) בין סכומי הקבוצות הוא המינימלי האפשרי.

השיטה משנה את המערך הבוליאני `fairDivision`, כך שהמערך הבוליאני מגדיר את החלוקה של מערך המספרים לשתי קבוצות (אם האיבר ה-`i` במערך הבוליאני הוא `true` אזי האיבר ה-`i` במערך המקורי שייך לרשימה הראשונה, אחרת שייך לרשימה השנייה).

דוגמאות:

- עבור המערך $\{1,2,3,4\}$ תתקבל התשובה $\{true,false,false,true\}$ או התשובה המתקבלת על ידי היפוך הסדר בין אותן שתי קבוצות: $\{false,true,true,false\}$. ההפרש המתקבל עבור החלוקה הנ"ל הוא 0.

- עבור המערך {1,2,7,17,6} יכולה להתקבל התשובה {true,true,true,false,true} או התשובה ההפוכה {false,false,false,true,false}. ההפרש המתקבל עבור החלוקה הנ"ל הוא:

$$\text{Math.abs}((1+2+7+6) - (17)) == 1$$

הקלות:

- לכל קלט יש לפחות שתי תשובות נכונות (התשובה הטובה ביותר ותשובה נוספת המתקבלת על ידי היפוך הקבוצות, כפי שהודגם קודם), יש למצוא ולהחזיר תשובה אחת.
- מותר להקצות פעם אחת (לא בשורה אחת אלא פעם אחת!) מערך עזר שהוא ליניארי בגודל הקלט.
- ניתן להניח ש-fairDivision ו-arr הינם מערכים בגודל זהה, ו-fairDivision מאוחזל כולו כ-false.

